

Video Streaming Protocols Comparison in Field Experiments for Aerial Teleoperation

Manuel J. Fernandez

*Department of Electrical Engineering,
Automation and Control
Technical University of Denmark
2800 Kongens Lyngby, Denmark
mjfgo@dtu.dk*

Riccardo Franceschini

*Robotics and Automation Unit
Eurecat Technology Center of Catalunya
Cerdanyola del Vallès, Barcelona, Spain
riccardo.franceschini@eurecat.org*

Julian Cayero Becerra

*Robotics and Automation Unit
Eurecat Technology Center of Catalunya
Cerdanyola del Vallès, Barcelona, Spain
julian.cayero@eurecat.org*

Matteo Fumagalli

*Department of Electrical Engineering,
Automation and Control
Technical University of Denmark
2800 Kongens Lyngby, Denmark
mafum@dtu.dk*

Abstract—As aerial robots evolve to perform complex tasks such as manipulation and visual inspection, maintaining the pilot at the center of decision-making is critical for mission safety and leveraging human expertise. A key enabler of this collaboration is reliable video streaming between the robot and the operator. This paper presents a comparative analysis of four video streaming protocols RTP (Real-time Transport Protocol), RTSP (Real-Time Streaming Protocol), SRT (Secure Reliable Transport), and ROSTCP (ROS Transport Control Protocol) based on extensive field experiments. The evaluation focuses on CPU usage, network throughput, latency, streaming quality, and packet integrity across varying distances. Tools such as *iperf*, *ping*, *GStreamer*, and *Wireshark* were employed for detailed performance measurements. Results reveal significant differences: SRT and RTP deliver superior network stability and resilience to packet loss, while RTSP and ROSTCP show advantages in CPU efficiency. These findings provide actionable insights for optimizing video streaming in aerial robotics, emphasizing the trade-offs and advantages of each protocol in real-world scenarios.

Index Terms—Human-Robot Interaction, Real-Time Communication, Teleoperation, Robotics

I. INTRODUCTION

Aerial robots are rapidly expanding their capabilities to perform a wide range of tasks [1], such as advanced manipulation techniques like drilling [2], locking onto pipes [3], or operating with fully compliant arms [4], [5]. They are also being equipped with advanced shared-autonomy algorithms for navigation [6] and infrastructure inspection [7], [8]. In all these operations, the operator is not only essential for ensuring the safety of the system and the environment but is also crucial for collaborating with the aerial robot, bringing their expertise into the decision-making loop. The operator's primary source of information is the live video feed from the UAV. Commercial solutions like DJI O3 [9] are commonly used for camera streaming; however, due to the proprietary nature of

such systems, they do not easily support the adaptation and customization to different platforms.

Selecting the appropriate video streaming protocol is therefore critical, as it significantly affects the quality of service, resource utilization, and overall user experience. Four widely used video streaming protocols are RTP (Real-time Transport Protocol) [10], RTSP (Real-Time Streaming Protocol) [11], SRT (Secure Reliable Transport) [12], and ROSTCP (ROS Transport Control Protocol) [13]. Each protocol offers unique features and benefits, but their performance can vary depending on the specific conditions of deployment.

This research aims to compare the performance of four video streaming protocols in a field experiment, where a remote camera streams video at varying distances. The study focuses on key performance metrics, including CPU consumption, network throughput, latency, streaming quality, and packet integrity. Data were collected using various tools: *iPerf* [14] for assessing network performance, *ping* for measuring latency, *GStreamer* [15] for evaluating streaming quality, and *Wireshark* [16] for packet analysis. The paper is structured as follows: Related Works (Sec. II) reviews previous comparisons to identify gaps in the research, while Methodology (Sec. III) details the experimental setup, both hardware and software. Results (Sec. IV) presents the findings of the experiments, and the Conclusion (Sec. V) summarizes the key insights and practical recommendations for optimizing video streaming in aerial robotics.

II. RELATED WORKS

Several studies have investigated the performance of video streaming protocols across various metrics such as latency, CPU consumption, network throughput, and packet integrity. For instance, [17] and [18] focus on comparing WebRTC and RTSP, two widely used protocols for real-time video transmission. These works provide valuable insights into the



Fig. 1: Map with locations logged during the experiment marked with the distance to receiver

communication paradigms and performance characteristics of both protocols, highlighting WebRTC's strengths in low-latency peer-to-peer applications and RTSP's robustness in client-server streaming architectures. However, these studies are largely confined to controlled environments and focus on protocol-level differences without accounting for key environmental factors, such as distance and real-world network variability, which can critically affect performance in on-field deployments.

While WebRTC and RTSP have been evaluated extensively, other relevant protocols, such as RTP, SRT, and ROSTCP, have not received the same level of comparative analysis in the context of real-time, field-based robotic systems. To the best of our knowledge, no comprehensive on-field study has been conducted that examines the performance of these protocols, particularly in robotic teleoperation scenarios, where the need for reliable and low-latency data transmission is critical for both operator decision-making and mission safety

A. Protocols Tested

In this work, we evaluate four video streaming protocols commonly used in real-time applications, each selected for its potential applicability to aerial robotic systems:

RTP (*Real-Time Transport Protocol*) [10]: is widely used in real-time communication systems such as VoIP and video conferencing. It supports the transmission of multimedia streams over a network and is known for its low-latency characteristics. In this experiment, RTP was tested for its ability to deliver stable, real-time video data under various network conditions, with a focus on minimizing delays.

RTSP (*Real-Time Streaming Protocol*) [11]: is a client-server protocol designed for establishing and controlling media sessions over a network. While often used in surveillance and

media-on-demand services like IP cameras, its performance in terms of resource efficiency and scalability was evaluated in this study, particularly in the context of long-range communications between an aerial robot and an operator. RTSP's suitability for structured, session-based streaming was also a point of focus.

SRT (*Secure Reliable Transport*) [12]: is a relatively new protocol, designed to provide secure and reliable video transmission over unpredictable networks. Its core feature, error correction, allows it to maintain video quality even over lossy networks, making it a candidate for aerial robotic applications where network stability can vary dramatically. This study tested SRT's performance in maintaining video quality over long distances, with an emphasis on packet recovery and minimizing video interruptions.

ROSTCP (*ROS Transport Control Protocol*) [13]: Primarily used in robotic systems, ROSTCP facilitates data and video transmission within the Robot Operating System (ROS) [19] framework. It is traditionally optimized for robot-to-robot communication but was included in this study to assess its viability as a video streaming protocol in field conditions, particularly for transmitting live video from aerial robots. Key metrics such as CPU load and latency were measured to evaluate its practicality in real-time operations.

III. METHODOLOGY

This section describes the procedure followed to setup and perform the field experiment. The goal of this experiment is to evaluate the performance of four protocols, by analyzing a set of metrics along known distances, represented in Figure 1, until reaching the limit in the communication range. The distances tested were from 0 meters to 300 meters, spaced by 50 meters, and a last one of 420 meters. The first one at 0 meters represent the transmitter next to the receiver. The last distance was set looking for the communication range limit.

A. Hardware Setup

The field experiment requires two setups (Fig. 2) to test each protocol along the range of communication. A base station, keeping a fixed position during the whole experiment, and a transmission station, placed at different distances from the receiver.

The base station consists on a laptop to receive the video streamed and a custom power station. The transmission station (Fig 3) is equipped with a Jetson Orin AGX [20] computer, an Alvium 1800 U-811 [21] camera and a battery. Both stations are connected to a wireless network interface.

The communication link has been chosen to provide a reliable communication between the remote device and the local computer, with a proper communication range to cover the operation. Thus, the network used in this experiment is a long-range Ubiquiti wireless Point-to-Point communication link. Base station uses an Ubiquiti Rocket AC [22] with a directional antenna UMA-D [23] and transmission station uses an Ubiquiti Bullet AC [24] with omnidirectional antenna Linx ANT-W63-IPW2-NP [25]. The maximum distance reachable

depends on the devices configuration and antennas. This experiment does not require to maximize this feature, so it is used a default configuration. This setup though not optimized, provides a consistent basis for comparing the video stream protocols and analyzing how the measured parameters are affected.



Fig. 2: Base (left) and transmission station (right)

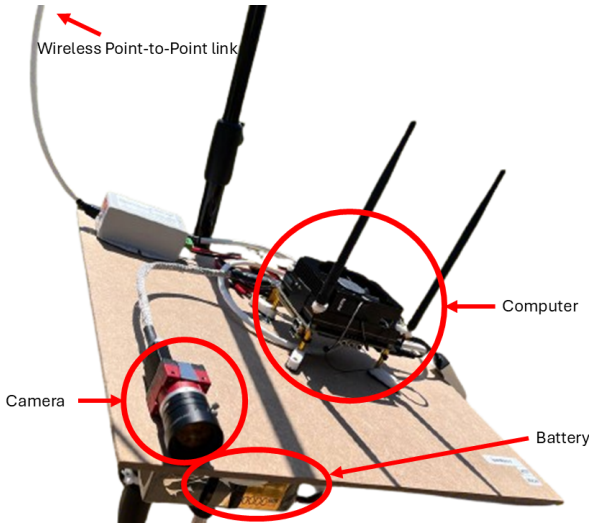


Fig. 3: Detail of transmission station setup

B. Software Setup

As Figure 4 shows, color images from the camera are published in a ROS topic using AVT Vimba SDK [26]. Then each frame is downsampled to 640x640 with a rate of 30Hz. To test ROSTCP protocol, images are compressed in *Compress node* using *image_transport* ROS package and visualized with the tool *rqt* at the receiver. The sender and receiver for the

rest of the protocols (RTP, RTSP and SRT) are implemented using a GStreamer [27] pipeline to configure each of them, reading the downsampled raw images from the camera and using H.264 [28] to encode them. This codec is one of the most widely used video compression standards. GStreamer is an open-source pipeline-based multimedia framework that allows to handle media source data for different purposes. RTSP sender is tested using the implementation from *ros_rtsp* [29], also developed using GStreamer. The GStreamer pipeline to send and receive the data is configured with same parameters for all the protocols.

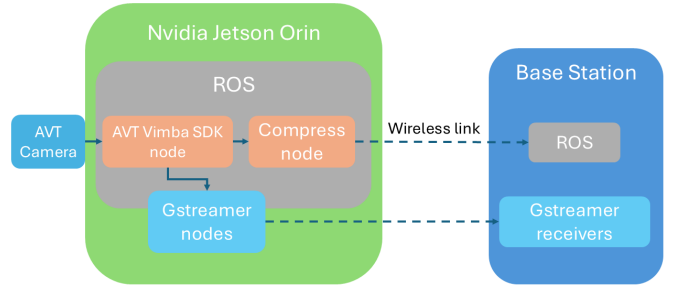


Fig. 4: General setup diagram flow

During the experiment some metrics are recorded to measure the performance of each protocol and the network quality. At each distance, the network quality is measured, before evaluating each protocol, between the two stations during 60 seconds using iPerf and Ping to get an estimation of the current Jitter, Round-Trip Time (RTT) and Packet loss. Then, each protocol is executed separately while data logs are recorded containing CPU load using the process viewer htop tool, latency at receiving registered with Wireshark and throughput and frame rate logged with GStreamer at the receiver. Throughput is measured at the beginning of the pipeline and frame rate at the end, just before showing the images on the screen. After the last protocol tested at each distance, all of them are executed at the same time to evaluate the streaming quality compared from a subjective point of view.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the results registered during the field experiment. As it is described in Section II-A, the set of protocols (RTP, RTSP, SRT, and ROSTCP) are evaluated along the range of communication limit, from 0 to 300 meters spaced 50 meters. All protocols behave properly without noticeable issues until 300 meters, when ROSTCP started to show delays while the rest kept a good performance. The streaming quality is subjectively similar during the next 50-100 meters so, aiming to register a log at the communication range limit, a last experiment is tested at 420 meters. This point is considered the limit distance since the connection is lost further away.

The range from this point to the disconnection is extremely short and the network quality drops quite aggressively.

Starting with the results from network quality measurements, Figure 5 shows RTT and Figure 6 plot packet loss and jitter. According with recommended guidelines to guarantee a QoS for interactive video [30], like IP videocalls, one-way delay should be no more than 150 ms, jitter should be lower than 30 ms, and packet loss below than 1%. These limits were added as reference to the plots.

RTT increases with the distance as it was expected, but it starts to increase when the receiver is really far. The values are small even at the limit of the communication range.

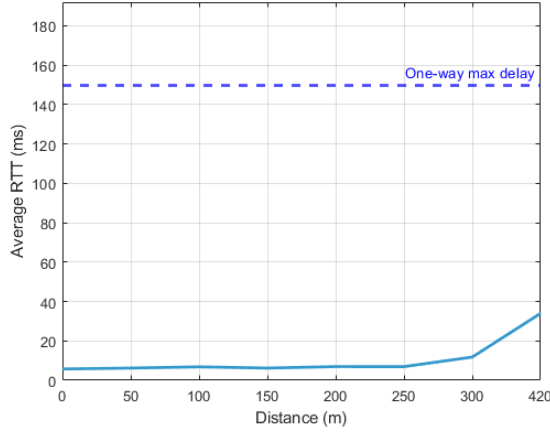


Fig. 5: Ping results for each distance - Average RTT and packet loss

Jitter measurements show very low values, so it indicates that the network latency is very stable along the whole range. The packet loss starts to be noticeable, but not critical, at the last two test but still below the recommended limit. A sample of packet loss in the frame showed in Figure 7, showing the effect over the video streaming.

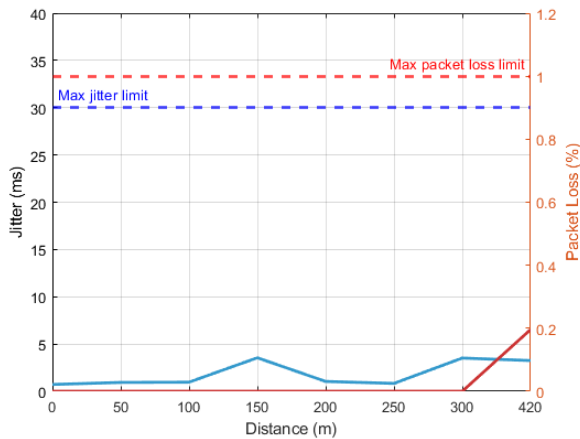


Fig. 6: Jitter measured for each distance



Fig. 7: Frame with packet loss issues

After evaluating the network performance, each protocol is evaluated separately. Table I shows the average CPU consumption for each protocol at different distances. Each 100% means the use of a full CPU core. Any value above implies the use of more than one CPU core. RTSP presents the higher load for the CPU across all distances compared with the others. ROSTCP shows the lowest CPU consumption. All protocols keep the same level of CPU load along the different distances logged.

TABLE I: Average %CPU load for each protocol

	ROSTCP	RTP	RTSP	SRT
0m	31.96%	100.25%	249.31%	100.70%
50m	31.11%	100.78%	202.33%	59.13%
100m	30.43%	95.05%	200.81%	85.39%
150m	30.25%	99.78%	233.91%	94.82%
200m	31.83%	101.09%	227.95%	81.49%
250m	31.48%	98.09%	194.47%	78.83%
300m	30.54%	99.21%	207.47%	91.31%
420m	29.15%	98.68%	221.47%	80.90%

A. Throughput and frame rate

Throughput was measured in terms of megabits per second (Mbps) using integrated tools of ROS for ROSTCP and Gstreamer for rest of protocols setting points to measure the performance of the pipeline. Table II presents the average throughput for each protocol at the different test distances with the frame rate achieved.

The results indicates that ROSTCP can not keep the frame rate at 300 and 420m. Regarding the throughput, RTP, RTSP and SRT demonstrate a more stable performance across the tested distances and also consume less bandwidth compared

TABLE II: Throughput (Mbps) / Frame rate (Hz)

	ROSTCP	RTP	RTSP	SRT
0m	20.16 / 30.005	0.8 / 29.51	2.27 / 29.62	0.84 / 29.73
50m	17.92 / 30.007	0.79 / 29.46	2.30 / 29.66	0.84 / 29.67
100m	14.56 / 30.040	0.8 / 29.46	2.28 / 29.65	0.84 / 29.69
150m	15.76 / 30.054	0.77 / 29.45	2.18 / 29.64	0.84 / 29.64
200m	23.04 / 29.987	0.79 / 29.51	2.32 / 29.62	0.82 / 29.76
250m	18.72 / 30.029	0.77 / 29.49	2.29 / 29.64	0.84 / 29.67
300m	9.68 / 18.053	0.79 / 29.41	2.30 / 29.63	0.85 / 29.49
420m	11.44 / 19.148	0.79 / 29.53	2.30 / 29.73	0.85 / 29.67

to ROSTCP. This may be due to the compression performed method used by ROS and the way images are transmitted. Figure 8 provide a detailed comparison of these protocols at 420m, where RTSP shows the most stable throughput over the time, despite consuming more bandwidth than RTP and SRT.

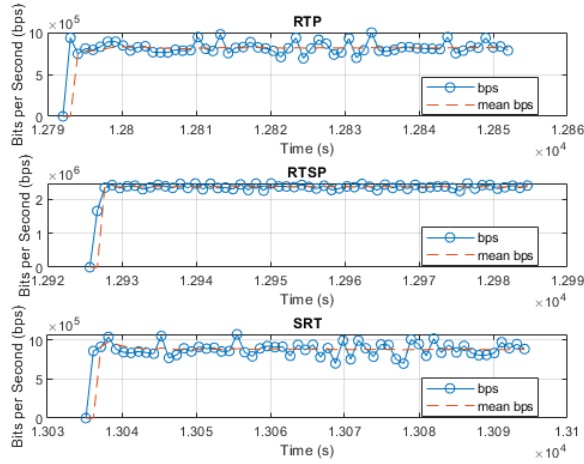


Fig. 8: Throughput logged with GStreamer

B. Latency on packets received

Table III shows the average latency of packets received for each protocol at each distance, keeping the focus on the time during the video transmission. Every protocols keep their values below the reference from [30] of 150ms as maximum one-way delay recommended.

TABLE III: Average latency (ms) on packets received

	ROSTCP	RTP	RTSP	SRT
0m	0.371	2.997	2.604	1.565
50m	0.368	3.001	2.335	1.562
100m	0.446	3.002	2.311	1.573
150m	0.416	3.004	2.507	1.562
200m	0.317	2.997	2.274	1.577
250m	0.358	2.985	2.278	1.554
300m	0.458	3.005	2.365	1.546
420m	0.839	3.004	2.277	1.604

Despite the low measurements from ROSTCP, it is the only protocol that shows an increase in its average value at the last distance. This results were obtained by considering every ROSTCP packet received, which suggest that the low values may be influenced by additional packets sent by ROS during the transmission among the video data.

C. Streaming Quality

For each distance, after every protocol is evaluated, all of them are executed at the same time to check the quality from a subjective point of view while the person who carry the device moves his hand in front of the camera. Streaming quality includes displaying a clear image without issues and keeping a reliable and video link. All protocols worked well until the streaming device reached the distance of 300 and 420m. Until

them, all of them behaved pretty similar, without noticeable delay or packet loss issues.

Figure 9 shows a frame once 300m was reached. The three on the top are RTP, RTSP and SRT from left to the right. ROSTCP, placed the bottom, started to delay noticeably more than the rest event getting small cuts occasionally. It is the worst as it was expected. RTP, RTSP and SRT continue performing well, showing very similar results. As the figure illustrates with the hand moving down, RTSP demonstrated the fastest performance followed by RTP and SRT. Executing them simultaneously, RTP and RTSP exhibit an almost identical behavior, while there is a slight difference with SRT.

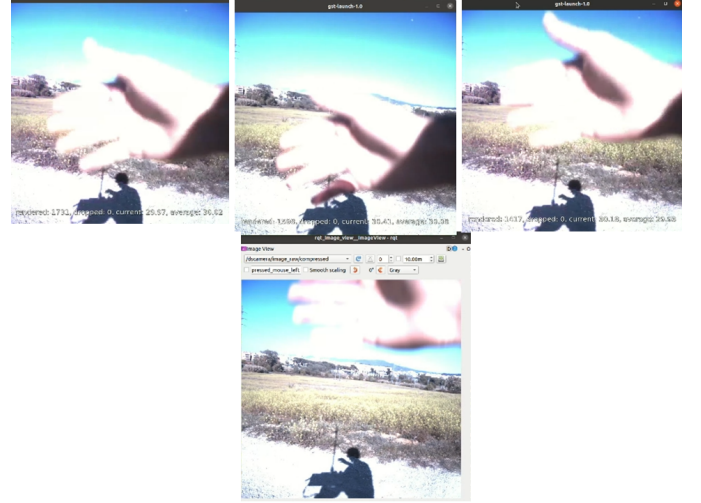


Fig. 9: Hand is going down - 300m

At the distance of 420m (Figure 10) all the protocols start to show issues, very noticeable in ROSTCP. RTSP still lead the performance, followed by RTP and SRT.



Fig. 10: Hand is going down - 420m

V. CONCLUSIONS

This study compared the performance of four video streaming protocols RTP, RTSP, SRT, and ROSTCP in a field

experiment designed to assess their efficiency across various distances. Considering the robotic teleoperation as use case, a long range Point-to-Point wireless communication was setup to cover the area to evaluate. This configuration presented a great signal quality along the whole range but with an aggressive signal drop in a very short range at the limit area.

By analyzing key performance metrics we identified significant differences in how each protocol handles real-world streaming conditions for a Point-to-Point configuration. RTSP performed well maintaining a consistent performance and provided the best quality, but it consumes more resources than others. RTP and SRT were close, keeping a good quality image and lower throughput and CPU load. ROSTCP, while shows the lowest CPU consumption, exhibited the worst performance when issues started to appear making it the worst choice if the remote device will work close to the limit of the communication range. RTSP was also the best during the recovering closely followed by RTP and SRT. It kept a an image quality with less errors than other.

VI. ACKNOWLEDGMENTS

This work has been supported by the European Unions Horizon 2020 Research and Innovation Programme AERO-TRAIN under Grant Agreement No. 953454 and by the Catalan Government through the funding grant ACCIÓ-Eurecat (Project TRAÇA – EUTFS).

REFERENCES

- [1] Anibal Ollero, Marco Tognon, Alejandro Suarez, Dongjun Lee, and Antonio Franchi. Past, present, and future of aerial robotic manipulators. *IEEE Transactions on Robotics*, 38(1):626–645, 2022.
- [2] Roman Dautzenberg, Timo Küster, Timon Mathis, Yann Roth, Curdin Steinauer, Gabriel Käppli, Julian Santen, Alina Arranhado, Friederike Biffar, Till Kötter, Christian Lanegger, Mike Allenspach, Roland Siegwart, and Rik Bähnmann. A perching and tilting aerial robot for precise and versatile power tool work on vertical walls. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1094–1101, 2023.
- [3] Jonathan Cacace, Giuseppe Andrea Fontanelli, and Vincenzo Lippiello. A novel hybrid aerial-ground manipulator for pipeline inspection tasks. In *2021 Aerial Robotic Systems Physically Interacting with the Environment (AIRPHARO)*, pages 1–6, 2021.
- [4] Alejandro Suarez, Guillermo Heredia, and Anibal Ollero. Design of an anthropomorphic, compliant, and lightweight dual arm for aerial manipulation. *IEEE Access*, 6:29173–29189, 2018.
- [5] Grigoriy A. Yashin, Daria Trinitatova, Ruslan T. Agishev, Roman Ibrahimov, and Dzmitry Tsetserukou. Aerovr: Virtual reality-based teleoperation with tactile feedback for aerial manipulation. In *2019 19th International Conference on Advanced Robotics (ICAR)*, pages 767–772, 2019.
- [6] Riccardo Franceschini, Javier Rodriguez Marquez, Matteo Fumagalli, and Julian Cayero Becerra. Riding the rollercoaster: Improving uav piloting skills with augmented visualization and collaborative planning. In *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 1093–1100, 2024.
- [7] Riccardo Franceschini, Javier Rodriguez Marquez, Matteo Fumagalli, and Julian Cayero Becerra. Point, segment, and inspect: Leveraging promptable segmentation models for semi-autonomous aerial inspection. In *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (ROMAN)*, pages 2204–2211, 2024.
- [8] Riccardo Franceschini, Matteo Fumagalli, and Julian Cayero Becerra. Enhancing human-drone interaction with human-meaningful visual feedback and shared-control strategies. In *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 1162–1167, 2023.
- [9] DJI. Dji o3 air unit, 2024.
- [10] Henning Schulzrinne, Steven Casner, R Frederick, and Van Jacobson. Rfc3550: Rtp: A transport protocol for real-time applications, 2003.
- [11] Henning Schulzrinne, Anup Rao, and Rob Lanphier. Rfc2326: Real time streaming protocol (rtsp), 1998.
- [12] Maria Sharabayko, Maxim Sharabayko, Jean Dube, Joonwoong Kim, and Jeongseok Kim. The SRT Protocol. Internet-Draft draft-sharabayko-srt-01, Internet Engineering Task Force, September 2021. Work in Progress.
- [13] TCPROS protocol. <https://wiki.ros.org/ros/tcpros>.
- [14] UDP iPerf The ultimate speed test tool for TCP and SCTP. <https://iperf.fr/>.
- [15] Gstreamer. Gstreamer, 2024.
- [16] Jay Beale, Angela Orebaugh, and Gilbert Ramirez. *Wireshark & Ethereal network protocol analyzer toolkit*. Elsevier, 2006.
- [17] Iván Santos-González, Alexandra Rivero-García, Tomás González-Barroso, Jezabel Molina-Gil, and Pino Caballero-Gil. Real-time streaming: A comparative study between rtsp and webtc. In Carmelo R. García, Pino Caballero-Gil, Mike Burmester, and Alexis Quesada-Arencibia, editors, *Ubiquitous Computing and Ambient Intelligence*, pages 313–325, Cham, 2016. Springer International Publishing.
- [18] Iván Santos-González, Alexandra Rivero-García, Jezabel Molina-Gil, and Pino Caballero-Gil. Implementation and analysis of real-time streaming protocols. *Sensors (Switzerland)*, 17, 4 2017.
- [19] Morgan Quigley, Brian Gerkey, Ken Conley, Josh Faust, Tully Foote, Jeremy Leibs, Eric Berger, Rob Wheeler, and Andrew Ng. ROS: an open-source Robot Operating System. page 6.
- [20] Nvidia Corporation. Jetson agx orin, 2024.
- [21] Allied Vision Technologies GmbH. Allied vision cameras, 2024.
- [22] Ubiquiti. Uisp airmax rocket 5ac lite basestation, 2024.
- [23] Ubiquiti. Directional dual-band antenna for uap-ac-m, 2024.
- [24] Ubiquiti. Dual-band airmax ac radio with dedicated wi-fi management, 2024.
- [25] Linx. Linx ant-w63-ipw2-np wifi 6 outdoor whip antenna, 2024.
- [26] Allied Vision Technologies GmbH. Vimba sdk for machine vision cameras, 2024.
- [27] Wim Taymans, Steve Baker, Andy Wingo, Ronald S Bultje, and Stefan Kost. Gstreamer application development manual (1.2. 3). *Publicado en la Web*, 72, 2013.
- [28] Hari Kalva. The h. 264 video coding standard. *IEEE multimedia*, 13(4):86–90, 2006.
- [29] ros_rtsp project repository. https://github.com/circusmonkey/ros_rtsp.
- [30] CISCO Systems. Enterprise qos solution reference network design guide.